

Adicionando informações de contexto de tarefa a uma ferramenta de chat síncrono para apoiar o desenvolvimento distribuído de software

Ana Paula Chaves
Coord. Tecn. em Sistemas
para Internet (COINT)
UTFPR – Campo Mourão - PR
anachaves@utfpr.edu.br

Igor Scaliante Wiese
Coord. Tecn. em Sistemas
para Internet (COINT)
UTFPR – Campo Mourão - PR
igor@utfpr.edu.br

Igor Steinmacher
Coord. Tecn. em Sistemas
para Internet (COINT)
UTFPR – Campo Mourão - PR
igorfs@utfpr.edu.br

Fabio Baia
UTFPR – Campo Mourão-PR
baiacfabio@gmail.com

Edilson F. da Costa
UTFPR – Campo Mourão-PR
edilsonfc@gmail.com

Karen Moreschi
UTFPR – Campo Mourão-PR
karenmoreschi@gmail.com

Cicero Jupi
UTFPR – Campo Mourão-PR
cicerojupi@gmail.com

Marco Aurélio Gerosa
Departamento de Ciência
da Computação(DCC-IME)
Univ. de São Paulo (USP)
gerosa@ime.usp.br

ABSTRACT

Communication is one of the challenges brought by the approach called Distributed Software Development (DSD). One way to improve communication is to make members of a conversation aware of other developer's context. This paper presents a synchronous chat tool to support DSD, by sharing the developer's context automatically. The tool is presented as an Eclipse plugin, and the information shared is related to the code piece in which the developer is working.

RESUMO

A comunicação é um dos desafios trazidos pela abordagem chamada Desenvolvimento Distribuído de Software (DDS). Uma das maneiras de melhorar a comunicação é possibilitar que, durante uma interação, os membros de uma conversa fiquem cientes do contexto da outra parte. Este artigo apresenta uma ferramenta de *chat* síncrono para apoiar o DDS, compartilhando o contexto da tarefa do desenvolvedor. A ferramenta é apresentada como um *plugin* para a IDE Eclipse, e as informações compartilhadas dizem respeito ao trecho de código em que o desenvolvedor está trabalhando.

Palavras-chave

Desenvolvimento Distribuído de Software, comunicação, contexto

1. INTRODUÇÃO

A indústria de desenvolvimento de software vem utilizando os benefícios trazidos pelo trabalho cooperativo apoiado por computador (CSCW) para obter as vantagens competitivas em termos de custo, qualidade e da possibilidade de encontrar profissionais em qualquer parte do globo [18]. Esta abordagem, chamada Desenvolvimento Distribuído de Software (DDS), é baseada em equipes geograficamente distantes trabalhando colaborativamente em um projeto de software. Entretanto, as vantagens são acompanhadas de desafios relacionados a diferenças culturais, contextuais, geográficas, organizacionais e políticas [13].

Dentre os desafios, destacam-se aqueles relacionados à comunicação, como a distância temporal e geográfica dos desenvolvedores [12]. Segundo Herbsleb [11], problemas na comunicação em DDS podem levar à falta de percepção e falta de informação sobre quem pode ajudar em uma tarefa, ou quem é responsável por executar uma tarefa. Isto acontece porque desenvolvedores em locais diferentes de trabalho percebem pequenas partes do contexto relacionado às tarefas e ao projeto em que estão trabalhando. Com isso, eles passam a desconhecer o contexto da tarefa dos outros membros e a disponibilidade para iniciar uma conversa. A falta de informação contextual dificulta o início de uma interação, pois a necessidade de compartilhar o contexto em que o problema será discutido pode tornar a comunicação um processo moroso, adiando a discussão do tópico principal.

Portanto, é necessário propor abordagens práticas para estabelecer comunicação a partir da percepção do contexto de uma tarefa. Este artigo apresenta uma ferramenta de *chat* síncrono que objetiva melhorar a comunicação por meio do compartilhamento de informações contextuais da tarefa em que o desenvolvedor está trabalhando. Tal ferramenta é apresentada na forma de um *plugin* para o ambiente de desenvolvimento Eclipse, evitando que o desenvolvedor necessite mudar o foco de atenção, e permitindo que o protocolo

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

Copyright 2011 ACM X-XXXXX-XX-X/XX/XX ...\$10.00.

de início da interação seja simplificado. As informações compartilhadas dizem respeito ao código fonte em que o desenvolvedor está trabalhando no momento. A ferramenta está em fase de desenvolvimento e um experimento está sendo conduzido a fim de avaliar seu uso.

O restante deste artigo está estruturado da seguinte forma: a Seção 2 discute a comunicação em DDS e apresenta alguns trabalhos relacionados; a Seção 3 apresenta a ferramenta de *chat* síncrono proposta para apoiar desenvolvedores em cenários de DDS; e a Seção 4 apresenta a conclusão deste artigo e os trabalhos futuros.

2. REFERENCIAL TEÓRICO

2.1 Comunicação em DDS

Membros de uma equipe de software podem colaborar de diferentes formas, tais como: chamadas telefônicas, fax, *instant messaging*, videoconferência e *email*. Segundo Paasivara [14], estas práticas são utilizadas para apoiar necessidades de comunicação em DDS: resolução de problemas, construção de relacionamento, tomada de decisão e coordenação. No entanto, alguns desafios precisam ser enfrentados como o design das ferramentas que não facilitam sua adoção [21]. Além disso, as ferramentas de comunicação não são aproveitadas pelos desenvolvedores em alguns casos. Muitas vezes, apesar da disponibilidade das ferramentas, os desenvolvedores não comunicam as alterações feitas durante o dia para outros membros da equipe dispersos geograficamente [16], resultando em repetição de trabalho e desperdício de tempo [17].

De acordo com Cockburn [5], as pessoas nem sequer sabem o que desejam comunicar na maioria dos casos. Assim, é importante pensar em integrar ferramentas ao contexto atual da tarefa, com o objetivo de evitar comunicação desnecessária e minimizar o risco de informações serem propagadas de forma incorreta ou incompleta.

Para Carmel et. al [3], a comunicação síncrona pode resolver falhas de comunicação, mal-entendidos e pequenos problemas antes que se tornem maiores. Um pequeno problema pode levar muito tempo para ser resolvido com discussões por meio assíncrono (e.g. *email*), mas uma breve conversa síncrona pode esclarecer rapidamente o problema. Assim, a comunicação assíncrona, muitas vezes pode gerar atrasos para a resolução de problemas. Como a codificação é uma tarefa dinâmica que necessita de resolução rápida, isto demanda interação entre os envolvidos, justificando a necessidade de meios síncronos para apoiá-la.

2.2 Trabalhos relacionados

Existem na literatura várias ferramentas e arquiteturas que tem como um de seus objetivos melhorar a comunicação para cenários de DDS, como observado em [23]. Uma ferramenta de *tickertape* (uma espécie de barra de *status* que exibe mensagens de eventos) foi introduzida em [8] para mostrar mensagens de *commits* em CVS (*Concurrent Version System*) para os membros de um projeto, possibilitando que estes iniciem um *chat* privado ou em grupo dentro do contexto da mensagem inserida no CVS. Communico [6] é uma ferramenta que apresenta mecanismos de percepção para possibilitar que participantes vejam discussões em andamento ou passadas, saibam quem está conversando e o nível de conhecimento desses indivíduos. Jazz [4] inclui uma ferramenta de *chat* para que desenvolvedores incluam *links*

para transcrições de *chats* anteriores e para as notificações de eventos das equipes. Uma extensão da ferramenta Jazz, chamada COFFEE [2], oferece uma ferramenta de discussão, incorporado na IDE Eclipse, possibilita que os desenvolvedores conduzam discussões categorizadas dentro do ambiente de desenvolvimento. EGRET [22] também oferece um *chat* contextualizado e ferramentas de *email* que facilitam a inserção automática ou manual de *links* navegáveis para artefatos relevantes nas mensagens.

Outros trabalhos ([7, 24, 1, 15]) apresentam ferramentas de *chat* síncrono como parte de ambientes de DDS mais completos, que oferecem diversas facilidades para equipes distribuídas. Porém as ferramentas de *chat* são fornecidas na forma de *instant messengers* tradicionais (como o Same-time, MSN, jabber), sem qualquer tipo de alteração que possa contribuir para tornar a comunicação mais fácil e mais efetiva em ambientes de DDS.

Existem alguns outros estudos com foco na fase de codificação, em especial para apoiar a prática de programação em pares em equipes distribuídas, possibilitando aos desenvolvedores cooperar sobre um determinado trecho de código ([9, 20, 19, 10]). Essas ferramentas geralmente oferecem formas dos desenvolvedores se comunicarem, exibindo seus papéis na interação e outras informações relevantes ao desenvolvimento em pares (como edição síncrona de código e compartilhamento de tela).

Os trabalhos apresentados apontam iniciativas para facilitar a comunicação em DDS e até mesmo para compartilhar o contexto em mensagens. Porém, essas iniciativas são geralmente integradas a ambientes mais complexos, cujo foco é mais amplo do que a comunicação em si. Foram identificadas outras pesquisas que tratam de mecanismos de comunicação para metodologias de desenvolvimento específicas, como é o caso do XP (*eXtreme Programming*). No entanto, não foram encontrados estudos que apresentem ferramentas específicas para comunicação, independentes de outras funcionalidades ou recursos, e que ofereçam o compartilhamento do contexto da interação de maneira simples.

3. FERRAMENTA PROPOSTA

Nesta seção, será apresentada a ferramenta proposta no presente artigo. Ela consiste em um *chat* síncrono para desenvolvedores, criado como um *plugin* para o ambiente de desenvolvimento Eclipse, como pode ser verificado na Figura 1. A ferramenta foi desenvolvida como uma evolução ao *plugin* EIMP¹, já existente no SourceForge.

A ferramenta apresenta-se como uma *View* do ambiente de desenvolvimento, e sua interface é semelhante à de um *chat* síncrono tradicional, conforme apresentado na Figura 2. Os contatos aparecem em uma lista ordenada por nome, apresentando o *status* de cada um deles. Até o momento, os *status* possíveis são desconectado (branco), ausente (amarelo), ocupado (vermelho) e disponível (verde).

A principal característica da ferramenta é possibilitar que, ao iniciar as interações, o contexto da tarefa do desenvolvedor seja automaticamente compartilhado com a outra parte. Isto é feito a partir do envio de uma mensagem automática no momento em que o *chat* é iniciado. Essa mensagem contém os nomes do projeto, da classe e do método (apresentando o local do código que está selecionado) onde o foco de trabalho do usuário que iniciou a interação está ativo

¹<http://sourceforge.net/projects/eimp>

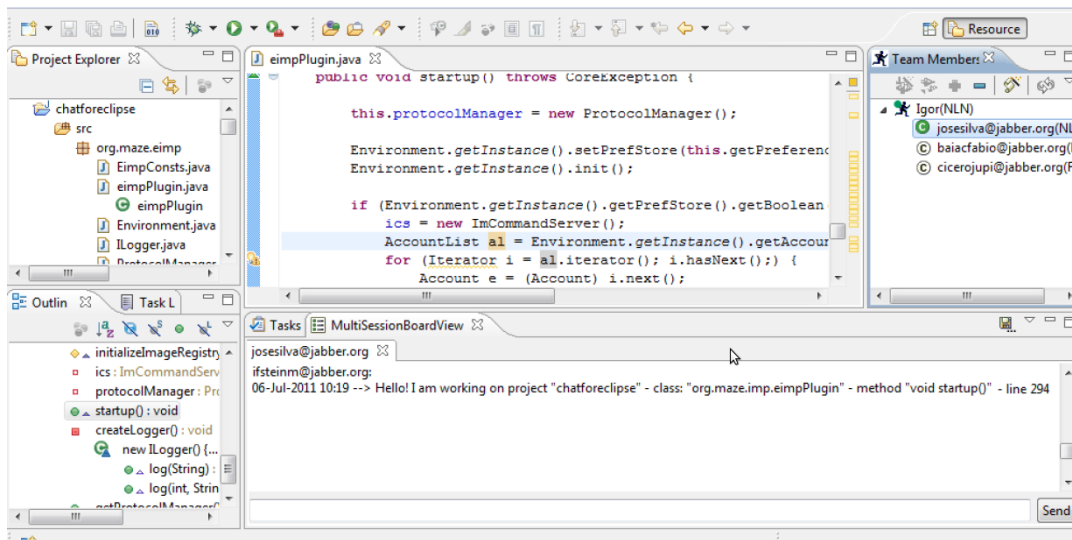


Figure 1: Ambiente Eclipse com a ferramenta de *chat* e uma mensagem contendo informações de contexto enviada automaticamente

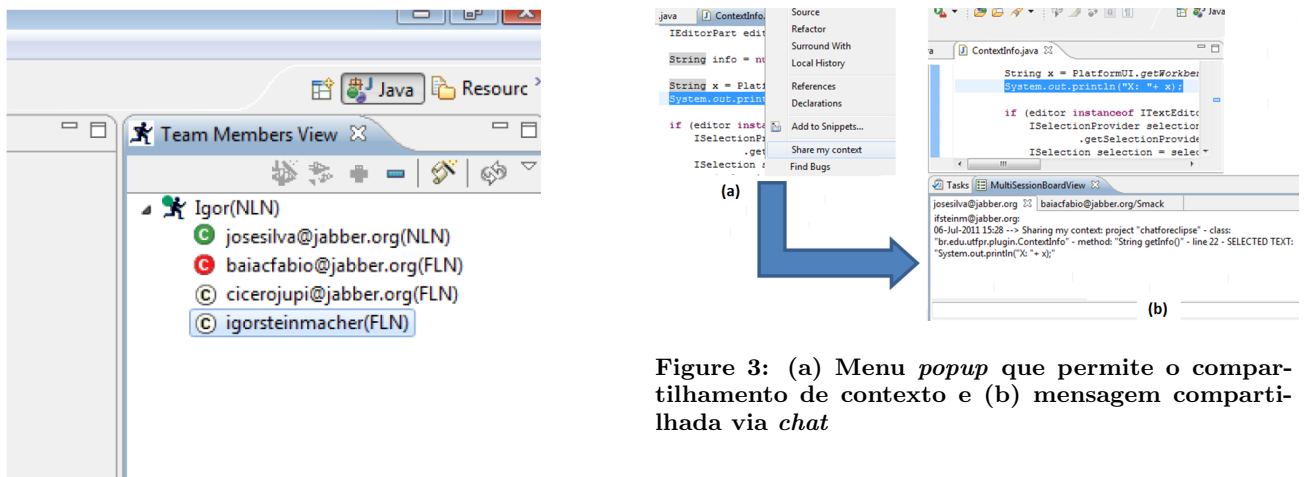


Figure 3: (a) Menu *popup* que permite o compartilhamento de contexto e (b) mensagem compartilhada via *chat*

Figure 2: Apresentação da lista de contatos e status

(Figura 1). Com essas informações sendo enviadas automaticamente, pretende-se que haja uma economia de tempo no início da comunicação. Isto deve ocorrer uma vez que são deixados de lado os passos iniciais do protocolo formal de comunicação, tornando a outra parte ciente do contexto da tarefa em que o desenvolvedor se encontra.

Durante a conversa, é possível solicitar o compartilhamento do local em que o desenvolvedor está focado. Isto é feito a partir de um menu *pop-up*, clicando na opção “Share my context”. Isto permite que, se solicitado, o desenvolvedor possa informar à outra parte o local exato em que está trabalhando. Neste caso, se um trecho de código estiver selecionado, o texto é compartilhado no *chat* (Figura 3).

4. CONCLUSÃO E TRABALHOS FUTUROS

O presente artigo apresentou uma ferramenta que tem por objetivo melhorar a comunicação por meio do compartilhamento de informações contextuais. Tal ferramenta é apre-

sentada na forma de um *plugin* para o ambiente de desenvolvimento Eclipse, evitando que o desenvolvedor necessite mudar o foco da atenção. O compartilhamento de contexto oferecido pela ferramenta visa, primordialmente, melhorar a comunicação e a coordenação entre membros de equipes geograficamente dispersas. Tal melhoria é obtida uma vez que o fluxo de comunicação é iniciado por uma mensagem padronizada, conhecida pelas partes envolvidas na conversa. Isso possibilita que as primeiras mensagens trocadas já digam respeito à tarefa em que os usuários estão trabalhando, tornando a comunicação mais rápida e efetiva.

Atualmente a ferramenta está em fase de desenvolvimento e está sendo utilizada pela equipe que realizou seu desenvolvimento. Estão sendo incorporadas novas funcionalidades e modos de apresentação das informações. Paralelo a isso, um experimento em ambiente acadêmico controlado está sendo preparado para verificar se a ferramenta atende aos objetivos propostos. Ao final do experimento, um questionário será aplicado aos participantes visando descobrir os reais benefícios e aplicações da ferramenta.

É importante observar que os dados compartilhados por meio da ferramenta não formam um conjunto exaustivo de

possíveis informações contextuais. Outros dados relacionados às tarefas do desenvolvedor podem ser utilizadas para aumentar a percepção dos membros da equipe sobre o trabalho colaborativo. Para tanto, faz-se necessário mapear as informações contextuais que necessitam ser compartilhadas para melhorar este cenário. Pretende-se, além dos dados sobre o código propriamente dito, coletar informações relativas ao relacionamento do desenvolvedor com o ambiente e com o projeto (por exemplo, tempo de interação com a classe, tempo que trabalha com o projeto, quantidade de contribuições etc.). Por fim, pretende-se, de acordo com as informações de contexto obtidas, recomendar interações com outros membros da equipe que melhor se adequem à situação.

5. REFERÊNCIAS

- [1] H. Bani-Salameh, C. Jeffery, Z. Al-Sharif, and I. Abu Doush. Integrating collaborative program development and debugging within a virtual environment. In *Groupware: Design, Implementation, and Use*, volume 5411 of *LNCS*, pages 107–120. Springer Berlin, 2008.
- [2] F. Belgiorno, I. Manno, G. Palmieri, and V. Scarano. Argumentation tools in a collaborative development environment. In *Cooperative Design, Visualization, and Engineering*, volume 6240 of *LNCS*, pages 39–46. Springer Berlin, 2010.
- [3] E. Carmel and R. Agarwal. Tactical approaches for alleviating distance in global software development. *IEEE Software*, 18(2):22–29, March 2001.
- [4] L.-T. Cheng, S. Hupfer, S. Ross, and J. Patterson. Jazzing up eclipse with collaborative tools. In *Proceedings of the 2003 OOPSLA workshop on eclipse technology eXchange*, pages 45–49, Anaheim, California, 2003. ACM.
- [5] H. Cockburn. *Agile software development*. Addison-Wesley Professional, Boston, MA, 2002.
- [6] K. Dullemond, B. van Gasteren, and R. van Solingen. Virtual open conversation spaces: Towards improved awareness in a gse setting. pages 247–256, aug. 2010.
- [7] R. Duque, M. L. Rodríguez, M. V. Hurtado, C. Bravo, and C. Rodríguez-Domínguez. Integration of collaboration and interaction analysis mechanisms in a concern-based architecture for groupware systems. *Science of Computer Programming*, 2010.
- [8] G. Fitzpatrick, P. Marshall, and A. Phillips. Cvs integration with notification and chat: lightweight software team collaboration. In *Proceedings of the 2006 20th anniversary conference on Computer supported cooperative work*, pages 49–58, Banff, Alberta, Canada, 2006. ACM.
- [9] B. Hanks. Empirical evaluation of distributed pair programming. *International Journal of Human Computer Studies*, 66(7):530–544, 2008.
- [10] R. Hegde and P. Dewan. Connecting programming environments to support ad-hoc collaboration. In *ASE 2008 - 23rd IEEE/ACM International Conference on Automated Software Engineering, Proceedings*, pages 178–187, 2008.
- [11] J. D. Herbsleb. Global software engineering: The future of socio-technical coordination. In *2007 Future of Software Engineering*, pages 188–198, Washington, DC, USA, 2007. IEEE Computer Society.
- [12] J. D. Herbsleb and D. Moitra. Guest editors' introduction: Global software development. *IEEE Software*, 18:16–20, 2001.
- [13] M. Ivceck and T. Galinac. Aspects of quality assurance in global software development organization. In *31st International Convention MIPRO*, pages 150–155, 2008.
- [14] M. Paasivaara. Communication needs, practices and supporting structures in global inter-organizational software development projects. 2003.
- [15] V. Penichet, J. Gallud, R. Tesoriero, and M. Lozano. Design and evaluation of a service oriented architecture-based application to support the collaborative edition of uml class diagrams. In *Computational Science - ICCS 2008*, volume 5103 of *LNCS*, pages 389–398. Springer Berlin, 2008.
- [16] R. Prikladnicki, J. Audy, and R. Evaristo. Distributed software development: Toward an understanding of the relationship between project team, users and customers. In *Proceedings of International Conference on Enterprise Information Systems (ICEIS)*, Angers, France, 2003.
- [17] V. Ramesh and A. Dennis. The object-oriented team: Lessons for virtual teams from global software development. In *System Sciences, 2002. HICSS. Proceedings of the 35th Annual Hawaii International Conference on*, pages 212 – 221, 2002.
- [18] M. Robinson and R. Kalakota. *Offshore Outsourcing: Business Models, ROI and Best Practices*. Mivar Press, Georgia, 2004.
- [19] S. Salinger, C. Oezbek, K. Beecher, and J. Schenk. Saros: an eclipse plug-in for distributed party programming. In *CHASE '10: Proceedings of the 2010 ICSE Workshop on Cooperative and Human Aspects of Software Engineering*, pages 48–55, Cape Town, South Africa, 2010. ACM.
- [20] T. Schuemmer and S. Lukosch. Supporting the social practices of distributed pair programming. In *Groupware: Design, Implementation, and Use*, volume 5411 of *LNCS*, pages 83–98. Springer Berlin, 2008.
- [21] N. S. Shami, N. Bos, Z. Wright, S. Hoch, K. Y. Kuan, J. Olson, and G. Olson. An experimental simulation of multi-site software development. In *Proceedings of the 2004 conference of the Centre for Advanced Studies on Collaborative research*, pages 255–266, Markham, Ontario, Canada, 2004. IBM Press.
- [22] V. Sinha, B. Sengupta, and S. Chandra. Enabling collaboration in distributed requirements management. *IEEE Software*, 23(5):52–61, 2006.
- [23] I. Steinmacher, A. P. Chaves, and M. A. Gerosa. Awareness support in global software development: a systematic review based on the 3c collaboration model. In *16th international conference on Collaboration and technology, CRIWG'10*, pages 185–201, Berlin, 2010. Springer-Verlag.
- [24] M.-A. D. Storey, D. Čubranić, and D. M. German. On the use of visualization to support awareness of human activities in software development: a survey and a framework. In *Proceedings of the 2005 ACM symposium on Software visualization*, pages 193–202, St. Louis, Missouri, 2005. ACM.